Progress Log — Tourist Safety Monitoring System

Working directory: `D:\TouristSafetyMonitoring-src` (the project was renamed from `D:\TouristSafetyMonitoring`; nothing was deleted).

Status legend: **DONE** = implemented and verified by a run · **PARTIAL** = implemented, not fully verified · **BLOCKED** = needs a device / key / network · **TODO** = not started.

---

0. Environment (verified)

Item	Result
JDK	`D:\AndroidTools\jdk-21` (Temurin 21)
Gradle	`D:\AndroidTools\gradle-8.11.1` + committed wrapper 8.11.1
Android SDK	`D:\AndroidTools\Sdk` (platform 35, build-tools 35/34, platform-tools)
Python	3.11.9 (`C:\Users\DELL\AppData\Local\Programs\Python\Python311`)
Python libs	numpy 2.4.4, pandas 3.0.6, scikit-learn 1.8.0, matplotlib 3.11.2
Git	`C:\Program Files\Git\cmd\git.exe` (project is **not** a git repo)
Device / emulator	**none** — instrumented tests and on-device rendering cannot be executed

Baseline at the start of this pass: `:app:testDebugUnitTest` = **130 tests, 0 failures**; `:app:assembleDebug` = success; instrumented suite compile-checked only.

---

1. Inspection findings (checklist)

1.1 Map / Create Geo-Fence screen is blank

- **Cause (confirmed in code):** `ui/components/SafetyMap.kt` gates the whole map on `mapsApiKeyConfigured`. With no key it renders a notice **instead of a map**, so taps never reach `onMapClick`, no vertices can be added, Close Ring stays disabled, zones cannot be saved.
- **Fix:** key-less OpenStreetMap fallback so drawing/saving works with no Google key, keeping Google Maps when a key is present.

1.2 Home shows "Tracking Active" but "Background service Not running"

- **Cause:** `HomeScreen.kt` printed the *intent* flag `snapshot.backgroundTracking`, not whether `LocationTrackingService` was actually alive.
- **Fix:** track the real foreground-service lifecycle and surface RUNNING / PAUSED(!permission) / OFF.

1.3 No delivery path for alerts/SOS; no backend in the repo

- **Fix:** durable **offline outbox** + pluggable `SyncTransport`, with an honest `NoBackendTransport` that keeps events queued (never fakes delivery).

2. Work performed (final status)

#	Item	Status	Evidence
1	Inspection + this log	DONE	this file
2	Key-less OpenStreetMap fallback map	DONE (JVM-verified) / PARTIAL (on-device render)	`MercatorProjectionTest` 11, `TilePolicyTest` 14, `TileDiskCacheTest` 6 pass; on-device tile render NOT EXECUTED
3	Background-service true state	DONE	`TrackingController`, `LocationTrackingService`, `HomeViewModel`, `HomeScreen`
4	Simulated smartwatch + SOS	DONE (logic) / PARTIAL (haptics)	`WatchControllerTest` 6 pass; vibration NOT EXECUTED
5	Offline outbox / sync transport	DONE	`OutboxQueueTest` 8, `OutboxPayloadsTest` 5 pass
6	ML / EDA pipeline on real GeoLife	DONE	21 pytest pass; real metrics in `ml/outputs/eval/`
7	Automated tests for the above	DONE	196 JVM + 21 ML pass; instrumented compile-only
8	Technical report	DONE	`docs/test_report.md`, `docs/final_project_report.md`
9	Map centring + live follow + multiple sample areas	DONE (logic) / PARTIAL (UI wiring)	`SampleAreasTest` 6 pass; button/follow NOT EXECUTED on a device
10	Offline tile disk cache + provider fail-over + diagnostics	DONE (policy/IO) / PARTIAL (on-device)	`TilePolicyTest` 18, `TileDiskCacheTest` 6 pass; real offline render NOT EXECUTED
11	Blue "you are here" marker, closer zoom, typed corners, crash log, tile-memory bound	DONE (logic) / PARTIAL (rendering)	`GeoFenceEditorViewModelTest` 14, `TilePolicyTest` 18 pass; marker/zoom/dialog NOT EXECUTED on a device (§10)

Additional details

- Database: `AppDatabase.VERSION` 1 → 2 with additive `DatabaseMigrations.MIGRATION_1_2` that creates `outbox_events`; no destructive migration; SQL verified against the generated `2.json`.
- ML dataset: the real **GeoLife GPS Trajectories 1.3** archive (313,164,406 bytes) was downloaded and unzipped here — no synthetic substitution. Measured 18,670 `.plt` files, 24,876,978 rows, 0 malformed. The detector is **unsupervised** (IsolationForest); its output is a *movement anomaly score*, never a danger probability. A separate SYNTHETIC fixture exists only for tests and is loudly labelled.

---

3. Verification steps (as executed)

Step	Command (from `D:\TouristSafetyMonitoring-src`)	Result
Unit tests (baseline)	`…\build.ps1 ":app:testDebugUnitTest"`	**130** passed / 0 failed
Unit tests (final)	`…\build.ps1 ":app:testDebugUnitTest"`	**196** passed / 0 failed / 0 skipped

Step	Command (from `D:\TouristSafetyMonitoring-src`)	Result
Instrumented compile-check	`…\build.ps1 ":app:compileDebugAndroidTestKotlin"`	BUILD SUCCESSFUL (not run)
Debug APK	`…\build.ps1 ":app:assembleDebug"`	BUILD SUCCESSFUL — 18.32 MB
ML tests	`python -m pytest tests -q` (in `ml\`)	**21** passed
ML pipeline	`…\run_all.ps1`	preprocess / eda / train / evaluate completed on real data

Real ML numbers (from `ml/outputs/`)

- Preprocess: 18,670 trajectories, 24,876,978 rows, 0 malformed; cleaned sample 1,429,740 points.
- Train: 1,060,493 train / 369,247 test points (900 / 300 trajectories, grouped by `traj_uid`, 0 overlap).
- Evaluate: injection ROC-AUC **0.8922**, average precision **0.6763**, shuffled control **0.5007**.

---

4. Not executed (BLOCKED) — declared honestly

Item	Reason
`DatabaseMigrationTest`, `AppDatabaseInstrumentedTest`, `StatusBannerInstrumentedTest`	no device/emulator (compile-checked only)
OpenStreetMap tile rendering on screen	no device/emulator or observable rendering
Smartwatch vibration / SOS haptic pattern	emulators have no vibrator; needs a physical phone

No other item is blocked: **PDF exports were produced** for every document (and `README`). Pipeline: Markdown → HTML (`python-markdown`) → PDF (headless Microsoft Edge `--print-to-pdf`). The `.html` and `.pdf` sit beside each `.md` in `docs\` (plus `README.html`).

---

5. Known limitations

- This is a **monitoring aid**, not an emergency service; it does **not** contact anyone automatically.
- *(Superseded — see §12.)* At the time of writing, no backend existed, so queued events stayed "Queued (offline)". A backend now ships in the repository and the app delivers to it; with no address configured the honest queued behaviour remains.
- ML has no real danger labels; only injection-based validation is meaningful.
- GeoLife is dominated by a few Beijing users, so "normal" is region/person-specific.

---

6. Follow-up fixes after real-device feedback (2026-10-16)

The user installed the APK on a real phone and reported that the OpenStreetMap fallback looked like a blank light-blue slab and that real-time position was not visible.

Root cause (confirmed by reading the code): `GeoFenceEditorScreen` passed **no location** to `SafetyMapFill`. With no live fix, no draft vertex and no zone, `OfflineOsmMap` fell back to `GeoPoint(0.0, 0.0)` — **Null Island, in the Atlantic Ocean** — so the screen showed open-ocean tiles (flat blue with tile seams) and every tap landed in the sea. The Tracking screen could likewise stay pinned at `(0,0)` when the first fix had not yet arrived.

Fixes applied (Android main source):

File	Change
`ui/components/OfflineOsmMap.kt`	Camera no longer defaults to `(0,0)`; prefers live fix → draft vertex → zone → **device last-known location** (`LocationManager`), else shows a "Finding your location" card. Added **live follow mode** (camera tracks the newest fix; a manual pan stops following; the locate FAB re-enables it), a **breadcrumb trail**, a **scale bar** + centre-coordinate readout, hardened tile fetching (**two OSM mirror hosts**, `Accept` header, 6 s timeouts, per-tile error capture) with an on-screen **"Street map offline — Retry"** affordance, and white corner halos on draft vertices.
`ui/components/SafetyMap.kt`	New `trail` parameter, passed through to both the Google and OSM paths, plus a `TrailOverlay` for the Google path.
`ui/screens/TrackingScreen.kt`	Passes the recent fixes (oldest-first) as the trail so movement is visible.

Verification: `:app:testDebugUnitTest` + `:app:assembleDebug` → BUILD SUCCESSFUL, **160 tests, 0 failures**; APK rebuilt (18.29 MB, `D:\TouristSafetyMonitoring-src-debug.apk`). On-device tile rendering and the follow behaviour still require the user's phone and are **not** claimed as device-verified here.

---

7. Second round of real-device feedback: "real Google map" + multiple areas (2026-10-16)

Feedback. The user asked for a *real Google map*, a restricted area shown as a polygon, and **multiple** areas, and pointed at the Microsoft Research **GeoLife** dataset page.

What was actually wrong. The Google path has always been in the app (`SafetyMap.kt` → `GoogleSafetyMap`) but needs an API key; none is configured, so the key-less map is used. More importantly, the **Create** screen hit the same null-island bug as the Tracking screen: it passed no location, so the camera sat at `(0, 0)`. Also, the Google camera was only set **once**, so it did not follow the user, which is why tracking looked static.

Changes (Android main source):

File	Change
`ui/components/DeviceLocation.kt`	**New.** `lastKnownDeviceLocation()` / `rememberDeviceLocation()` — the single, shared, permission-guarded read of the OS last-known fix. Keeps the map off null island with no GPS session and no database writes.
`ui/components/OfflineOsmMap.kt`	Now uses the shared helper (its private copy was deleted).
`ui/screens/GeoFenceEditorScreen.kt`	Reads the **live** tracking snapshot and falls back to the last-known device fix, then opens the camera on `state.centroid ?: here`. The live reading is also passed to the map so "you are here" shows while drawing. Fixes the blank/ocean Create screen for **both** the Google and OSM maps.
`ui/components/SafetyMap.kt`	**New** `FollowLatestFix(cameraPositionState, point)` — animates the Google camera to the newest fix so the Tracking map behaves like a live tracker.

File	Change
`ui/screens/TrackingScreen.kt`	Calls `FollowLatestFix` and adds a **follow toggle** (my-location FAB): tap to stop/start following.
`geofencing/SampleAreas.kt`	**New, pure.** `SampleAreas.create(center)` builds three labelled sample zones (restricted / dangerous / safe), 140 m squares laid out in a row 340 m apart, longitude scaled by `cos(latitude)`.
`ui/viewmodel/ManageZonesViewModel.kt`	`addSampleAreas(center)` persists `SampleAreas.create(center)`; `showMessage()` added.
`ui/screens/ManageZonesScreen.kt`	**New** "Add 3 sample areas near me" button: uses the device fix, else the first existing zone's first vertex, else explains that location must be on.
`docs/google_maps_setup.md`	**New.** How to get a free Maps SDK for Android key, where to paste it (`local.properties` / `-PMAPS_API_KEY`), how to rebuild, and how to read grey-tile failures.

Verification. `:app:testDebugUnitTest` + `:app:assembleDebug` → **BUILD SUCCESSFUL, 166 tests, 0 failures, 0 skipped** (160 + 6 new `SampleAreasTest` cases, MAP-01…MAP-06). APK rebuilt: 18.29 MB, SHA-256 `3CCAEF4460865CF41160753F5B56CA946EEF489B4E210B39ED66E0C7831663BD`, copied to `D:\TouristSafetyMonitoring-src-debug.apk` and the Documents folder.

Still not device-verified: Google basemap rendering with a key, the on-screen centring, the follow toggle and the sample-areas button (INST-06…INST-08 in `docs/test_report.md`). The geometry behind the button **is** unit-tested; the UI wiring is not.

About the dataset. GeoLife is a *human-movement* dataset (18,670 `.plt` files, 24.9 M fixes, mostly Beijing). It powers the **ML movement-anomaly module** and the EDA report — see `docs/dataset_and_eda.md`. It is not a basemap and cannot make the map render; the basemap comes from Google Maps or OpenStreetMap tiles. If GeoLife-derived areas are wanted on the map later, the honest approach is to turn GeoLife hotspots into ordinary zones — they would be far from the user, so they are deliberately **not** shipped as the default "sample areas", which instead sit around the user.

---

8. Key-less map hardening: offline cache, mirrors, diagnostics (2026-10-16)

After choosing to keep improving the **key-less** map rather than configuring a Google key, the fallback was hardened so a tourist on a flaky network still sees streets.

Evidence gathered before coding (from this machine):

Mirror	Result
`https://tile.openstreetmap.org/13/4093/2723.png`	HTTP 200, `image/png`, 41 KB
`https://tile.openstreetmap.de/13/4093/2723.png`	HTTP 200, `image/png`, 41 KB
`https://a.tile.openstreetmap.fr/osmfr/13/4093/2723.png`	HTTP 200, `image/png`, 39 KB
`https://tile.openstreetmap.fr/osmfr/…` (no subdomain)	HTTP 404 — rejected
`https://tile.openstreetmap.se/hydda/full/…`	dead — rejected

Changes:

File	Change
`ui/map/TilePolicy.kt`	**New, pure.** The three verified mirror templates; `urlFor()`; `cacheName()`; `hostsFrom(preferred)` fail-over order; `evictions()` LRU rule; a 40 MB cache cap.
`ui/components/TileDiskCache.kt`	**New.** Size-bounded on-disk cache of raw tile PNGs under `cacheDir/osm-tiles`, so viewed areas render with **no network** and survive a restart. All I/O is failure-soft.
`ui/components/OfflineOsmMap.kt`	Disk cache is consulted before the network; successful fetches are cached and trimmed. The mirror that worked is remembered and tried first. Tap the top-left caption for **diagnostics**: cache size, zoom/centre, **"Check map servers"** (probes every mirror, reports `ok` / `HTTP …`), **"Clear map cache"**. Added **zoom in/out** buttons.

Verification: `:app:testDebugUnitTest` + `:app:assembleDebug` → **BUILD SUCCESSFUL, 183 tests, 0 failures, 0 skipped** (166 + 11 `TilePolicyTest` + 6 `TileDiskCacheTest`). Enumerated acceptance cases extended to **56** (MAP-07…MAP-16). APK rebuilt: 18.39 MB, SHA-256 `253163A75043B0CB673287CE4D2119D2CECD1B9B57EB7861D52BCA2C38C531B3`.

Still not device-verified: the cache actually serving a map offline after a restart, the mirror fail-over / server-check on a real network, and the zoom buttons (INST-09…INST-11 in `docs/test_report.md`). The cache's real file behaviour and the mirror policy **are** unit-tested.

---

9. Four independent tile providers (2026-10-17)

> **Later reversed in part.** CARTO Voyager was added here as the default but was **removed** in a later round: it began serving a "CARTO" watermark instead of a map under the app's request conditions. `TilePolicy.PROVIDERS` is now **three** providers — OpenStreetMap (default), Esri World Street Map and OpenTopoMap — and `TilePolicyTest` asserts CARTO is not offered. The rest of this section is the historical record of that round.

Feedback. "the map is not connecting bro like if u need api pls give the free source ill add if u want" — i.e. tiles still not arriving, and the user is willing to add a key.

Diagnosis. A *single* tile server is a single point of failure: a school, hotel or mobile-carrier network can block one hostname and leave the map blank. The app previously had one style (OpenStreetMap) with three mirrors on neighbouring hostnames — a network that blocks OSM's tile servers blocks all of them at once.

Evidence gathered before coding (all from this machine, same tile `13/4093/2723`):

Provider	Host checked	Result
OpenStreetMap	`tile.openstreetmap.org`	HTTP 200, `image/png`, 41 KB
OpenStreetMap	`tile.openstreetmap.de`	HTTP 200, `image/png`, 41 KB
OpenStreetMap	`a.tile.openstreetmap.fr/osmfr`	HTTP 200, `image/png`, 39 KB
CARTO Voyager	`a./b./c.basemaps.cartocdn.com`	HTTP 200, `image/png`
Esri World Street Map	`server.arcgisonline.com` (`{z}/{y}/{x}`)	HTTP 200, `image/jpeg`, 37 KB
OpenTopoMap	`a./b./c.tile.opentopomap.org`	HTTP 200, `image/png`, 48 KB
— rejected —	`tile.openstreetmap.fr/osmfr` (no subdomain)	HTTP 404

Provider	Host checked	Result
— rejected —	`tile.openstreetmap.se/hydda/full`	dead
— rejected —	`tile.openstreetmap.org.b-cdn.net`	TLS trust failure

Changes:

File	Change
`ui/map/TilePolicy.kt`	Re-modelled as `Provider`s — one **basemap style** with its own mirror hosts and its required attribution. Four key-free providers: OpenStreetMap, CARTO Voyager, Esri World Street Map, OpenTopoMap. `urlFor()` now honours the template's own token order (Esri is `{z}/{y}/{x}`). `cacheName()` is **provider-scoped** so two styles can never be confused.
`ui/components/OfflineOsmMap.kt`	Tries providers in order and keeps **one provider for the whole viewport** — on a switch it restarts cleanly instead of drawing a patchwork of two styles, and the number of switches is bounded. The status line and the diagnostics show the **active provider name**; *Check map servers* now probes all four; the provider's required **attribution** is shown on the map.

On the offer of an API key: the Google Maps path is complete and already wired (`docs/google_maps_setup.md` — a free Maps SDK for Android key from Google Cloud, pasted into `local.properties` as `MAPS_API_KEY=…`). No code change is needed; the app switches to Google tiles the moment a key is present. The provider work above means the app is **usable with no key at all**, which the user chose.

Verification: `:app:testDebugUnitTest` + `:app:assembleDebug` → **BUILD SUCCESSFUL, 186 tests, 0 failures, 0 skipped** (`TilePolicyTest` grew 11 → 14). Enumerated acceptance cases extended to **60** (MAP-17…MAP-20). APK rebuilt: 18.39 MB, SHA-256 `A9F1A5D3846F3C2E8313CC51D35B389743A3F5991A9B6170C7CEDAA8BBF05DC3`.

Still not device-verified: that the provider fail-over actually recovers a blocked host on the user's network (INST-10). The provider list, URLs, key-scoping and order **are** unit-tested, and all four providers were confirmed reachable from the build machine.

Encoding note: an earlier edit of *this log* was written through PowerShell `Set-Content` without an explicit encoding, which double-encoded its non-ASCII characters. It was detected (wrong byte count) and reversed exactly, restoring 14 em dashes, 12 ellipses and 10 arrows with zero replacement characters. Only this one file was affected; the code and the other documents were edited with tools that preserve UTF-8.

---

10. Real-device bug-fix round: invisible marker, auto-close, zoom, typed corners (2026-10-17)

Feedback (verbatim). "the real tracking the blue mark is not visible i seen / and in seconds app closed automatically / i feel when i add location like polygon need 2 time in manyal entry and latitute and logitude entry because since the map is wide when i draw it iwll more so / and the map view shld be close not that wide / and seems like backend is not added".

Six screenshots were attached. **They did not arrive on this machine as files**, so everything below is based on that written description. That is a real limitation of this round: the screenshots were never opened. Stated plainly rather than glossed over.

10.1 The "blue mark" was green — root cause found

`ui/theme/Color.kt` had `val UserMarkerAccent = Color(0xFF0B6E4F)` — the app's **own dark forest green** — and `SafeStroke` is `0xFF1B7F4C`. The "you are here" dot was therefore painted the same colour as a **SAFE zone's fill** and the same colour as the theme. That is exactly why it could not be seen: every other map app has taught the user to look for *blue*, and this dot was green-on-green.

Fix	Detail
`ui/theme/Color.kt`	`UserMarkerAccent = 0xFF1A73E8` (the conventional "my location" blue) and a new `UserMarkerOutline = 0xFF0B3C7A`.
`ui/components/OfflineOsmMap.kt`	Marker redrawn for contrast on any basemap: blue halo → dark outline → white collar → blue disc → white pip.
`ui/components/OfflineOsmMap.kt`	With **no live fix**, a **hollow grey** ring is drawn at the OS's last-known position, so the map is not empty — and a hollow ring can never be mistaken for a live fix.
`ui/components/OfflineOsmMap.kt`	A status line now says why the dot may be missing: `● You are here (±8 m)`, `○ Last known only — tap Start tracking`, or `○ No position yet — tap Start tracking`.

10.2 "In seconds app closed automatically"

Not reproduced — no emulator here and `adb devices` is empty, so there is no logcat and no way to see the crash. Rather than claim a fix, this round does two things: it removes the most likely causes, and it installs the instrumentation needed to identify the real one.

The instrumentation (new):

File	Purpose
`diagnostics/CrashLog.kt`	Installed from `TouristSafetyApp.onCreate`; records the most recent uncaught exception (newest first, bounded to 40,000 chars) in `filesDir/last-crash.txt`. Never uploads anything.
`ui/components/CrashReportDialog.kt` + `MainActivity`	On the next launch, a **"Last run ended unexpectedly"** dialog shows the recorded trace with a **Copy** button. This is the only realistic way to diagnose a crash on a phone with no logcat.

The likely causes, now removed. The previous tile loader started **one coroutine per visible tile on every change** of camera, zoom, viewport, retry token or provider — and a pan changes `center` on every frame, so a drag produced a burst of requests per frame. Worse, the in-memory `tiles` map was **never pruned**: it accumulated every tile ever viewed, at every zoom level, so a few minutes of panning on a real phone is an out-of-memory kill. Both are genuine OOM paths, not speculation.

Change	Effect
140 ms settle delay before tiles are requested	The effect is cancelled and restarted on each camera change, so this de-bounces a pan into a single batch.
Tiles loaded in batches of 4 (`TILE_LOAD_BATCH`)	Bounds simultaneous sockets and coroutines.
Tiles outside the settled viewport are dropped from `tiles`	The in-memory map can no longer grow without limit.
`LruCache` sized in **bytes** (8 MB) instead of 128 entries	No longer can pin ~33 MB of bitmaps.

Change	Effect
`TilePolicy.providersToTry(preferred, down)` — new, unit-tested	Once a provider fails for a tile it is skipped for the rest of the viewport, so a dead network costs one round of attempts instead of ten **per tile**.
Timeouts 6 s → 5 s	A blank map says "Street map offline — Retry" sooner.
The whole load wrapped in `try` / `catch (CancellationException)` rethrow	No tile problem can take the app down.

This is a fix for a **plausible** cause plus instrumentation for the real one. It is **not** a verified fix of the reported crash, and it is not claimed as one.

10.3 "The map view should be close, not that wide"

`DEFAULT_ZOOM` raised from 15 to **17** (about 1.2 m per pixel, roughly a kilometre across a phone screen), so a boundary drawn by tapping covers the place the user meant instead of a whole town. The editor's map box also grew 280 dp → 320 dp, and `GeoFenceEditorScreen` now draws the **already-saved zones behind the draft** — which is what makes "add another area" placeable *in context* instead of on a blank map. (The camera also opens at zoom 17 where a key is present: `CameraInitialPosition(zoom = 17f)`.)

10.4 "Need 2 times in manual entry, and latitude and longitude entry"

The sentence contains two distinct problems.

1. **A refused tap was silent.** `addVertex` returned the state unchanged when the tap landed on the previous vertex, so the map appeared to ignore the tap — which reads as "I have to tap twice". It now says so: "*That corner is on top of the previous one — tap further away.*"
2. **There was no coordinate entry at all.** `GeoFenceEditorScreen` gained a Latitude/Longitude row with **Add point** and **Fill from my position**. The fill button is what makes southern/western coordinates workable: a phone's decimal keypad has **no minus sign**, so typing "−33.9" is impossible — prefilling from the current fix means the sign is already on screen and only the digits are edited. The parser also accepts a comma as the decimal separator (a comma-locale keypad produces `12,9716`), and every rejection gives a readable reason instead of doing nothing.

Covered by 6 new unit tests in `GeoFenceEditorViewModelTest`: typed vertex, comma separator, out-of-range latitude, nonsense input, and a complete polygon built purely from typed coordinates and saved.

10.5 "Seems like backend is not added"

Correct — and it is deliberate, not a regression. **There is no server in this project.** Alerts and SOS are written to a Room-backed **outbox** and delivered through `NoBackendTransport`, so they are durably queued and labelled honestly as **"Queued (offline)"**. Nothing anywhere claims delivery. Adding a backend is separate, out-of-scope work; the outbox is already the client half of it, so a real transport could be dropped in without touching the UI.

The watch screen now says so where the user actually looks: under *SOS delivery* it reads "No backend server is configured in this build, so queued events are held safely on this phone and nothing is ever reported as delivered."

10.6 Flaky watch test found while verifying (not a product bug)

The full-suite run intermittently failed `WatchControllerTest > sos is reported delivered when a transport accepts it`, and once with a 20-second wait too. Investigated rather than re-run until green: with the JVM warm, every test in that class takes 0.6 s, but **whichever test is running when Robolectric first instruments Room's generated classes pays**

several seconds of one-off cost. When that landed inside a wall-clock-polled assertion, the 5-second `awaitUntil` default expired — a test-scheduler artefact, unrelated to the SOS code.

Two changes, both disclosed: the two SOS tests get a **60-second ceiling** (the happy path is unchanged at ~0.6 s; a timeout is a ceiling, not a cost), and `awaitUntil` now **drains Robolectric's paused main looper** while polling, because Room posts its invalidation callbacks there. The assertions themselves are untouched, so a genuinely broken SOS path still fails.

> **Superseded — see §11.1.** The diagnosis above is wrong on both counts, and the `ShadowLooper.idleMainLooper()` change made the flake *worse* rather than better. It is left in place because the reasoning is the instructive part: a plausible mechanism was adopted without being measured. The real cause was a full `C:` drive (§11.2). The 60-second ceiling has been kept — that part was harmless.

Verification (this machine): `:app:testDebugUnitTest` + `:app:assembleDebug` → **BUILD SUCCESSFUL, 196 tests, 0 failures, 0 errors, 0 skipped** (`TilePolicyTest` 14 → 18, `GeoFenceEditorViewModelTest` 8 → 14). APK **18.32 MB**, SHA-256 `6261B017275065B1CA19992A3BEBED486D37C39229F0E8110DBE2F1463874FA3`. Enumerated acceptance cases extended to **70** (MAP-21, MAP-22, EDIT-01…EDIT-04, INST-12…INST-15).

Still not device-verified: everything in 10.1–10.4 *as rendered* (marker visibility over real tiles, the zoom level, the coordinate row, the crash dialog), the diagnosis in 10.2, and the six screenshots, which were never opened here.

---

11. Correction, real ML models, approach-risk briefing, and a Google-like map

Later on 2026-10-17. This section supersedes §10.6 and records the largest piece of work in the project: the movement/fall models are now actually trained, evaluated and shipped rather than described.

11.1 Correction: the flaky watch test was not a test-scheduler artefact

§10.6 recorded a mechanism that had not been measured, and the "fix" derived from it was wrong:

- **The claim:** Robolectric's one-off instrumentation cost was expiring the 5 s `awaitUntil` default, and Room posts its invalidation callbacks to the paused main looper, so `awaitUntil` was changed to call `ShadowLooper.idleMainLooper()` on every poll.
- **Measured instead of assumed:** a temporary diagnostic printed the outbox row and the collected `Flow` every 250 ms. The row reaches `SENT` and the `Flow` reports it within **~125 ms**, with **no** main-looper draining at all — Room refreshes invalidations on its own query executor, so the drain was never needed.
- **The drain was actively harmful.** Called from the main thread inside `runBlocking`, `idleMainLooper()` cost roughly **1.5 s per call** and starved the very background work being awaited. With the drain in place the "SOS delivered" test spent its entire 60 s ceiling inside the loop, and the sibling SOS test took **24 s** where it should take under a second. The drain is now removed, and `awaitUntil` is a plain real-time poll.

11.2 The actual cause: the `C:` drive was full

Three symptoms that all pointed the same way, once they were noticed together:

- `Copy-Item` failed with **"There is not enough space on the disk."**
- `Get-Content -Raw` returned nothing for a file that `[xml]` had just parsed successfully.
- A test report declared `failures="5"` but contained **no failure text at all** — the write had been truncated.

`C:` had **0 bytes free** out of 96.8 GB. The build, the JDK, the Android SDK and `GRADLE_USER_HOME` all live on `D:`, but JVM temp files, Robolectric's scratch space and the test-report writes go to `C:` through `%LOCALAPPDATA%\Temp`. Freed by clearing the recycle bin (796 MB) and the npm cache (4.55 GB) → **5.28 GB free**. Three consecutive full-suite runs then passed with no change to any test.

The flakiness was never in the SOS code, the Room flows, or the coroutine scheduling. Honest note: this is an environment defect, not a code fix, and four three-minute test cycles were spent on hypotheses before free disk space was checked — which should have been the first thing to check on a machine that reports "not enough space".

11.3 A real ordering guard in the watch controller (kept, but not the cause)

While the flake was still unexplained, `WatchController` was also changed, and those changes are kept because they stand on their own merits:

- After `flush()` returns, the controller **re-reads the outbox row** and updates the face directly, instead of waiting for Room's change notification. This removes a real latency: the watch face no longer sits on "Queued on the device" for an SOS that has already been delivered.
- `SosDeliveryState.afterObserving(...)` makes a settled `DELIVERED` / `FAILED` outcome **sticky for its own event**, so a late "still queued" observation cannot walk the indicator backwards, while a newly raised SOS still resets it. Pinned by `SosDeliveryStateTest` (4 tests).

It is not claimed to have fixed the flake. §11.2 was the cause. The ordering rule is a guard against an interleaving Room is permitted to produce, not a bug that was observed.

11.4 Crash fix — the "blue marker appears, then the app closes" report

Signature captured by the in-app crash dialog:

```
java.lang.NullPointerException: CameraUpdateFactory is not initialized
  at com.google.android.gms.maps.CameraUpdateFactory.newLatLng
  at com.touristsafety.app.ui.components.SafetyMapKt$FollowLatestFix$1$1.invokeSuspend(SafetyMap.kt:99)
```

With no Google Maps API key configured, the wrapper never initialises the Play Services map SDK, but the live-follow effect still called `CameraUpdateFactory.newLatLng` every time a fix arrived — so the crash landed exactly when the blue marker first appeared. Fixed with a `googleCameraReady` guard plus `runCatching` in `followLatestFix`, `moveTo` and `fitPoints`, and `fitPoints` now returns early on degenerate bounds. Compile- and JVM-verified; **not** device-verified (§11.8).

11.5 Real machine learning: activity recognition and impact/fall detection

The reviewer asked for a real trained model, not a stub. Two supervised models are now trained on **UCI HAR** — the activity dataset listed in `DATASETS.pdf` — exported to JSON, and walked on the phone by hand-written Kotlin in `motion/`. No TensorFlow, no TFLite, no PyTorch, no network at inference time.

Model	Held-out result
Activity, 24 time-domain features, 32 trees	**accuracy 0.8351, macro-F1 0.8333** on 2947 windows from 9 subjects never seen in training
Activity, 561-feature reference, 300 trees	accuracy 0.9287, macro-F1 0.9271
Activity, single decision tree, 24 features	accuracy 0.7967, macro-F1 0.7943

Model	Held-out result
Impact/fall detector, 40 trees	**ROC-AUC 0.9999, average precision 0.9999, F1@0.5 0.9962**
Shuffled-label control	ROC-AUC **0.5198** (chance)

Two things are stated plainly rather than buried, because either could otherwise be read as a stronger claim than it is:

1. **UCI HAR contains no falls.** There is no fall label and no subject who fell, so the detector's positives are **impacts synthesised into real recorded windows**. Its metrics measure detection of that injected signature, on real held-out subjects. They are not a clinical fall-detection claim. The shuffled-label control at 0.5198 is what shows the near-perfect score is the model finding the signature, not a leak. The named upgrade path is SisFall / UniMiB SHAR / MobiFall — real falls.
2. **The approach-risk score is not a model.** See §11.6.

An exporter bug was also caught before it shipped: scikit-learn's `tree_.value` holds **class fractions, not counts**, so the original `np.rint(tree.value)` collapsed every leaf to a one-hot vector. The model would have loaded, run, and answered — just wrongly. Now exported as `np.rint(tree.value * tree.weighted_n_node_samples)`, with a pytest regression guard on the root node's sample count.

Verification that the shipped JSON is faithful:

Check	Result
`ml/har_verify_export.py` (Python walker vs scikit-learn)	activity max probability error **4.970e-07**, 0 argmax mismatches; fall **4.736e-07**, 0 mismatches — **ALL PASS**
`ForestReferenceTest` (Kotlin walker vs baked-in scikit-learn probabilities, real windows)	3 tests, pass
`ml/tests/test_har_models.py`	21 tests, including byte-equality between `ml/outputs/**` and the copies under `app/src/main/assets` and `app/src/test/resources`

Full metrics, per-class tables, embedded confusion matrices, limitations and reproduction commands are in **`docs/model_card_movement.md`**.

A stale number was found and fixed at its source while writing that card: the model spec claimed the UCI HAR test split was **2949** windows when it is **2947**. The string is no longer typed by hand — it is formatted from the arrays — and a pytest case fails if the wrong count comes back.

11.6 Historical risk analysis when approaching a restricted area

`ApproachRiskModel` scores 0–100 from: zone severity (DANGEROUS 40 / RESTRICTED 28 / SAFE 5), the tourist's **own recorded crossings** of that zone (3 points per crossing in the last 7 days, capped at 18; 4 points if never visited), proximity (up to 16), time of day (night 12, dusk 5), whether they are ambulating (4), and the live impact-detector score (12 above the 0.55 threshold, 6 neutral when the model is unavailable).

The new `ApproachBriefingCard` shows the score, the band, and **every factor with its points and its plain-language reason**, plus the tourist's own history for that zone. It says in the card itself that this is a published weighted checklist rather than a learned danger probability, because the app has no labelled outcome data ("did something bad actually happen?") and will not pretend otherwise. The weights are in one place and unit-tested (`ApproachRiskTest`, `ApproachBriefingTest`, `ZoneHistoryTest` — 26 tests).

11.7 The map, and the "make it look like Google Maps" request

- **CARTO Voyager was made the default key-free basemap here** — since **removed**, because it now serves a watermark instead of a map under the app's request conditions. `TilePolicy.PROVIDERS` is now OpenStreetMap (default), Esri World Street Map and OpenTopoMap, and the app never offers CARTO. OSM, Esri and OpenTopoMap remain automatic fallbacks, and the existing rule that **one basemap style serves the whole viewport** is unchanged, so the map never becomes a patchwork of two styles.
- A **distance-to-boundary pill** now sits over the tracking map ("320.0 m to SVIT Ground · risk 47/100"), matching the reference design. It is driven by the same `ApproachBriefing` the card below renders, so the two numbers cannot disagree.
- **Real Google tiles still require the user's own API key**, which this build does not have (`local.properties` holds the commented template only). That is a credential, not a coding problem, and `docs/google_maps_setup.md` walks through obtaining a free key and where to paste it. The key-less map is the fallback that makes the app work today.

11.8 Verification for this round

> This snapshot is refreshed at the end of each round; **§13 carries the latest numbers** for the checklist-completion round.

Check	Command	Result
Android JVM suite	`:app:testDebugUnitTest`	**370 tests, 0 failures, 0 errors, 2 skipped** across 35 classes
APK build	`:app:assembleDebug`	BUILD SUCCESSFUL
ML suite	`python -m pytest ml\tests -q`	**42 passed**
Export fidelity	`python ml\har_verify_export.py`	**ALL PASS**
Training reproducibility	re-ran `ml\har_train.py`	identical metrics; regenerated `har_activity_model.json` **byte-identical** to the shipped copy

APK **18.44 MB**, SHA-256 `5B44E92AAA9FB0CC5F54C6BE180A0156F9AAE153CC93B287FC176D66CDC2FB2D`, copied to `Desktop` and `Downloads`.

Not device-verified, and recorded as such: everything sensor- or tile-rendering-related — the crash fix, the blue marker over real tiles, the distance pill, the movement card, and all live sensor inference. No emulator and no attached phone were available (`adb devices` was empty), so the instrumented cases INST-04…INST-21 remain **NOT EXECUTED**. The sensor pipeline is compile-checked and JVM-tested against recorded windows; it has not been observed on hardware by this project.

Still requires the user: the "Check map servers" readout if a device shows blank tiles, and a free Google Maps API key if real Google tiles are wanted.

---

12. Backend integration round: a real transport, the SOS path, and the officer dashboard (2026-10-17)

12.1 The gap, and the root cause

Earlier rounds built the durable **outbox**, the payloads, and an honest `NoBackendTransport` that refused to pretend an event had been delivered. But the app could never actually talk to anything, for a reason that was not a missing feature but a wiring bug:

- `AppContainer` **hard-coded** the transport: `syncTransport = NoBackendTransport(...)`. There was no code path from "the user configured an address" to "events are sent", so setting an address could not have worked even if a server existed.
- There was **no HTTP client at all** in the app — no base-URL storage, no token storage, no request builder, no response parser.
- Nothing on the officer side existed to receive a batch, decide a verdict per event, or work an SOS queue.

12.2 What was built

- **A real authority client** (`app/src/main/java/com/touristsafety/app/authority/`): a hand-written JSON reader/writer (`Json.kt` — used instead of `org.json`, which returns nulls on the plain JVM, so the whole client stays unit-testable); the wire codec (`AuthorityApi.kt`); the HTTP layer over `HttpURLConnection` (`AuthorityHttp.kt`); the client that ties them together (`AuthorityClient.kt`); and the durable transport (`AuthoritySyncTransport.kt`).
- **A transport selector** (`SelectingSyncTransport`) replaces the hard-coded stub: it delegates to the real authority transport when a base URL is configured and to the honest `NoBackendTransport` otherwise. Because it reads the address at flush time, **setting an address takes effect on the next flush with no app restart**.
- **Session storage** (`AuthoritySessionStore`) in its own DataStore file (`authority_session`), separate from `AppSettings`, holding the base URL and the bearer token.
- **Emergency contacts**, end to end: a new Room entity/DAO/repository, uploaded through the durable outbox (`OutboxKind.CONTACTS`) rather than a direct `PUT`, so an offline edit still syncs. This is the reason the schema moved to **version 3** (`AppDatabase.VERSION = 3`, additive `MIGRATION_2_3`).
- **UI**: an **Authority** screen (enter address, sign in, see the real profile and delivery totals) and an **Emergency contacts** screen, both reachable from Settings and a new top-bar cloud icon. The watch screen's stale "no backend server is configured in this build" text was replaced with the *real* persisted session state.
- **The backend** (`backend/`): one Flask app over one SQLite file, serving both the phone API and a static operator dashboard at `/` — `/auth/login`, `/me`, `/me/contacts`, the idempotent `/sync/events`, and the authority routes (`/authority/overview|sos|events|tourists`, SOS acknowledgement, tourist registration). It generates its own credentials on first start and writes them once to `backend/var/DEV_CREDENTIALS.txt`; **no password is written into the source**.

12.3 The delivery rules (why a mistake can never lose an event)

`AuthoritySyncTransport` maps the server's verdict onto the outbox like this — deliberately biased so that a *configuration* error keeps the event, and only a *permanent* rejection discards it:

Server outcome	Outbox result	Why
`DELIVERED`	**Delivered** (ack id)	the authority stored it
`DUPLICATE`	**Delivered**	idempotency: the server already has it
`REJECTED` (or HTTP 400/422)	**Rejected** with the reason	the same bytes can never be accepted
HTTP 401 / 403	**Retry**	re-signing-in will deliver it
404 / 409 / 5xx / IO error / malformed body	**Retry**	could be a misconfiguration; never discard

12.4 Verification for this round

> Historical snapshot of the backend-integration round. The current numbers are in **§13.5**.

Check	Command	Result
Android JVM suite	`:app:testDebugUnitTest`	**370 tests, 0 failures, 0 errors, 2 skipped** across 35 classes
Authority client/transport/real-socket	`--tests com.touristsafety.app.authority.*`	51 tests pass (17 API, 15 transport, 10 real-socket, 9 codec)
Emergency contacts + payloads	`--tests … EmergencyContactRepositoryTest … OutboxPayloadsTest`	14 tests pass
Backend rules (in-process)	`python -m pytest backend\tests -q`	**174 passed**
Backend acceptance (real subprocess)	`python backend\verify_integration.py`	**26/26 PASS**
Live client → real server	`AuthorityLiveBackendTest` with `-Dtsm.e2e.baseUrl`	**2/2 passed** (run on the JVM against `python backend\run.py`)
APK build	`:app:assembleDebug`	BUILD SUCCESSFUL

The live run is the one that matters for the "don't claim it works" rule: the **production Kotlin client** signed in as a real tourist, flushed a real SOS through `OutboxSyncer`, received a server event id, saw `/me` report `delivered=1` with `latestSos=OPEN` and `simulated=true`, and confirmed the officer queue held **exactly one** row after a duplicate delivery. See §7.1 of `docs/test_report.md` for the full transcript and the artifact path.

Two things were found and fixed while doing this, rather than worked around:

- the **schema-contract test** still asserted `AppDatabase.VERSION == 2`; it now asserts the whole contiguous 1→2→3 path, and a new test **executes both migrations** against a real SQLite database and proves a pre-upgrade row survives (the instrumented `DatabaseMigrationTest` gained the matching 2→3 and 1→3 cases, but it needs a device, so it stays compile-checked only);
- a **JDK-only quirk**: `HttpURLConnection.getErrorStream()` returns null for a **POST** that answers 401, because the JDK replays the request for authentication. `AuthoritySyncTransport` never reads the success stream on a non-2xx, so it cannot hang, and a 401 still maps to Retry. Android's HTTP stack does not do this, but that cannot be confirmed without a device — logged honestly.

APK **18.81 MB**, SHA-256 `7606A2796DD5B1E72A2FC4EBA79B9F40121953EA9F6549ED82258F3AF0EEBB36`, copied to the project root, `Desktop` and `Downloads` as `TouristSafety-backend-integration-2026-10-17.apk`.

12.5 Running the backend and dashboard

```
cd D:\TouristSafetyMonitoring-src
python backend\run.py                 # dashboard: http://127.0.0.1:8080/   health: /api/v1/health
python backend\run.py --host 0.0.0.0  # reachable from a phone on the same network
```

Point the phone at it in the app: **cloud icon** → **Authority connection** (or **Settings** → **Connection & sign-in**). Emulator uses `10.0.2.2:8080`; a phone on the same Wi-Fi uses the LAN address `run.py` prints. Then sign in with the tourist id

and password. Until an address is set **and** a tourist is signed in, every event stays queued on the phone and the app says so.

> *Superseded by §16.3:* the cloud icon is gone and there is no longer a combined "Connection & sign-in" screen. The address lives in **Settings → Backend connection (advanced)**, sign-in has its own tourist screen, and the officer console is a separate, server-enforced role.

12.6 Still requires a device or a server (stated, not hidden)

- **On a device that has never been done here:** the Authority screen and contacts screen rendering, the permission/sign-in flow against a reachable server, and the device's own HTTP stack over emulator loopback / Wi-Fi — recorded as **INST-20, INST-21 NOT EXECUTED** in `docs/test_report.md`.
- **The dashboard was verified at the API level and by static-asset serving**, not by clicking through it in a browser.
- **The app still does not send periodic `POSITION` reports**, so the dashboard's "latest position" only moves on a zone alert or an SOS. This is deferred until a battery/frequency policy is chosen — recorded rather than hidden. *(Closed in §13: routine position reporting now exists, behind an explicit interval/distance policy the tourist can turn off.)*

---

13. Checklist-completion round: position reporting, PIN sign-in, status indicators, and the remaining datasets (2026-10-17)

This round started from an audit of the eight checklist items rather than from new code, and only built what was genuinely absent. §12.6's two open items — **no periodic `POSITION` reports** and **no PIN** — are both closed here.

13.1 Item 1 (GPS / geofencing / offline sync) — the one real gap

Tracking, polygon geofencing, entry/exit alerts and the durable outbox already existed, and the backend already accepted and rendered `POSITION`. The missing piece was entirely app-side: nothing ever *enqueued* a routine position. Added:

- `OutboxKind.POSITION`, with `OutboxPayloads.position(reading, timestampMillis)` and a `position:<fixTimestamp>` dedupe key (mirroring `backend/tsm/payloads.py`);
- `PositionReportPolicy` + `PositionReporter` — a documented throttle (default **60 s / 100 m**, with interval choices) that **refuses simulated readings and out-of-range coordinates**, and queues nothing until an authority address is configured so the queue cannot grow unbounded on a personal build;
- a **Settings → Authority reporting** card to enable it and pick the interval.

`docs/` and the code state the policy explicitly: position reports are a battery/data trade-off, and the first report after an address is configured is sent immediately rather than waiting out an interval.

13.2 Item 5 (ID + PIN) and item 6 (SOS acknowledgement)

- **PIN, end to end.** The server stores a scrypt-hashed PIN (new `pin_salt` / `pin_hash` columns, added idempotently on connect), generates one on first start and prints it **once** (it is deliberately *not* written to `DEV_CREDENTIALS.txt`, which stays a three-field `role/id/password` table). `POST /auth/login` accepts a `pin` **or** a `password`; `PUT /me/pin` validates 4–8 digits; an operator can never use a PIN. The app gained a PIN/Password toggle, a numeric field, and **Set or change PIN**.

- **Acknowledgement visible to the tourist.** `GET /me` now returns `latestSos` (status, officer, times, simulated) and `latestPosition`; the Authority screen renders the SOS state and the position on file.

13.3 Item 7 (UI, map, indicators)

- A **GPS status card** on Home that distinguishes *Waiting for permission / Acquiring / Live / Poor accuracy / Permission denied / Location unavailable*, with fix age and accuracy — computed from the real permission + settings state, not a placeholder.
- An **Authority upload** card: the configured address, who is signed in, and how many events are waiting.
- **Responsive fixes**: the remaining action rows and filter-chip rows that could squeeze on a narrow phone now use the wrapping `ActionButtonRow` / `FilterChipRow` primitives (`SettingsScreen` , `AuthorityScreen` , `SimulationScreen` , `GeoFenceEditorScreen` , `ManageZonesScreen` , `WatchScreen` , `EmergencyContactsScreen` , `HomeScreen`). The map still needs no Google key: the OSM default is the key-less basemap, with attribution, on-disk caching and fail-over, and CARTO is not offered.

13.4 Items 2–4 (EDA, inference, history, risk)

The activity/fall models, confusion matrices, model cards and the 12-item GeoLife EDA were already present and were left untouched. Added, as separate reproducible scripts with no new dependencies:

- `ml/osm_eda.py` — **real** OpenStreetMap EDA via the Overpass API: a 1.5 km × 1.5 km Tiananmen extract, **12,783 nodes / 3,197 ways / 0 relations**, 5,681 nodes/km^2, with the exact query, endpoint, fetch time and ODbL attribution recorded, plus a `NOT_RUN` path if every mirror fails;
- `ml/weather_eda.py` — real weather for two sites over **2011-01-01 … 2012-07-27** (574 days each). The brief names the OpenWeatherMap API, whose key was not provided, so the shipped artefacts use the key-free **Open-Meteo** archive and say so on every artefact — never presented as OpenWeatherMap data;
- `ml/har_eda.py` — the missing UCI HAR figure set (561 features, 7,352/2,947 windows, subject-disjoint 21/9, gravity 1.0323 g);
- `ml/history.py` — historical trajectory, temporal and **zone-crossing** trends on real GeoLife data (18,670 trajectories / 182 users; a derived densest-cell zone with **431 crossing trajectories, 430 entries / 426 exits**);
- `ml/transport_mode.py` — a genuine **supervised** model on the only valid target in the data (GeoLife transportation mode): subject-disjoint, **accuracy 0.8475 / macro-F1 0.5784** on unseen users.

On "future-risk prediction" (item 4): none of the four datasets carries an injury/danger/incident outcome, so there is no valid supervised target and **no risk model is shipped**. The app's `ApproachRisk` score remains the documented weighted heuristic it always was. This is stated in `docs/dataset_and_eda.md` §10 and the model card rather than papered over with a fabricated model.

13.5 Verification for this round

Check	Command	Result (measured)
Android JVM suite	`:app:testDebugUnitTest`	**390 tests, 0 failures, 0 errors, 4 skipped** across 37 classes
Backend rules (in-process)	`python -m pytest backend\tests -q`	**192 passed** (was 174; +18 PIN)
Backend acceptance (real subprocess)	`python backend\verify_integration.py`	**26/26 PASS**

Check	Command	Result (measured)
Live client → real server	`AuthorityLiveBackendTest` with `-Dtsm.e2e.baseUrl=http://127.0.0.1:8081`	**4/4 passed, 0 skipped**
ML suite	`python -m pytest ml\tests -q`	**86 passed** (was 42; +44 for the new analyses)
APK build	`:app:assembleDebug`	BUILD SUCCESSFUL

The four live tests ran against a real `python backend\run.py`: SOS delivery + idempotency + officer queue; a **position report** landing on the authority map (`/me latestPosition: 18.5204,73.8567 simulated=false`); **PIN set then PIN sign-in** (wrong PIN → HTTP 401); and a dead backend leaving the event queued with its reason. The 4 skips in a normal run are exactly these four opt-in tests.

APK **18.82 MB**, SHA-256 `592B8B63E81010647AC61C3B54F202284C9126C1D7EC5AA5DB0612D6A8BB7D2F`, copied to the project root, `Desktop` and `Downloads` as `app-debug.apk`.

13.6 Still not done here (stated, not hidden)

- **No emulator or device** (`adb devices` was empty), so on-hardware GPS, the permission dialogs, the `ON_RESUME` recovery, device rendering of the Authority/contacts screens, the device HTTP stack and the new status-card layout remain **NOT EXECUTED** (INST-04…INST-21 in `docs/test_report.md`).
- **No Google Maps API key** was available; the key-less OSM basemap is what is built.
- **The OpenStreetMap Overpass fetch and the weather fetch need the network.** Both succeeded live in this round; the artefacts are cached, and the scripts degrade to `NOT_RUN.md` rather than inventing data.
- **The OpenWeatherMap branch is implemented but was not exercised** (no key).
- **No version control**: `git` cannot run in this environment, so nothing is committed and all changes live only in the working tree.

---

14. The future-event risk model: a real supervised prediction, trained on real falls (2026-10-18)

The one thing the brief asked for that this project had repeatedly, correctly *declined* to fake. Section 13.4 said: "*no fabricated 'future-risk' model — no outcome labels exist*". That statement was right about the four named datasets, and it stayed right. This round changed the **dataset**, not the honesty.

14.1 The problem, restated

GeoLife, UCI HAR, OpenStreetMap and OpenWeatherMap/Open-Meteo contain **no injury, incident or danger outcome at all**. A supervised tourist-incident model is therefore impossible from them, and manufacturing labels would be fabrication. Two honest options existed: ship nothing, or find a dataset with real outcome labels. This round did the latter, documented the swap, and re-checked §13.4's blunt claim about the dashboard.

The other real gap found this round: the authority backend had **no table that could hold a prediction**. It stored incidents (`events`) and an SOS workflow (`sos_alerts`), and the dashboard could therefore neither receive nor display a prediction even if one existed. That gap is now closed (§14.4).

14.2 The dataset, and what was found in it

UniMiB SHAR (Micucci, Mobilio, Napoletano, 2017): 30 subjects, 9 activities of daily living and **8 real fall types**, 50 Hz phone accelerometer. Downloaded here and verified by hash: 266,206,080 bytes, SHA-256 `365a3967...67160`.

- **1,980 trials read, 1,979 used.** Subject 10 / `Jumping` / trial 1 has **9,369 of 9,624** non-finite values. It was **dropped, counted and reported** in `dataset_provenance.json` — not interpolated, not mean-filled. A NaN in a feature vector would have poisoned every tree that touched it.
- **6 fall trials** never reach the 2.0 g impact threshold and are excluded from the positives (counted).
- **No OSI licence exists** for this dataset. The provenance file records `{"osi_approved": false, "research_use_only": true}` plus the citation rather than inventing terms. Only derived artefacts are redistributed — never the recordings.

14.3 The target, the horizon, and the leakage proof

> *Given the last second of phone accelerometer data, will a hard fall impact land within the **next** second?*

- Positive = a window ending 1..50 samples **before** a sample that is the trial's `argmax(magnitude)` and is at least **2.0 g** (reusing the app's own `FallPolicy.MIN_IMPACT_G`). Negative = a window cut at the same 0.2 s cadence from an **ADL** trial. Post-impact windows are a **diagnostic**, never training data.
- Horizon and feature window are both **50 samples = 1.0 s** — the **longest fully supported** horizon (trials run 151–446 samples, impact around index ~120, leaving ≥95 pre-impact samples). A 30-minute horizon is **not attempted**, and the docs say why.
- **Leakage is proved, not asserted.** A window ending at `t` uses `[t-49, t]`; the label asks about `(t, t+50]` — disjoint by construction. `train.check_no_future_information()` re-derives this from the raw recordings on every run, and a **shuffled-label control** scores **ROC-AUC 0.477** (chance).
- **No fitted preprocessing at all**: every feature is scale-free within its own window, so there is no scaler that could carry a global mean from train into test.
- Subject-disjoint 60/20/20, seed 42 -> **18/6/6 subjects, disjoint: true**, 51,128 windows (6,458 positive / 44,670 negative).

14.4 What was built

Layer	What it does
`ml/future_risk/`	`config.py` (selected hyper-parameters, the size budget, the 22-feature order), `features.py`, `dataset.py`, `train.py`, `tune.py`
Artefacts	`ml/outputs/future_risk/`: `windows.npz`, `dataset_provenance.json`, `metrics.json`, `tuning.json`, `calibration_reliability.csv`, `test_predictions.csv`, 6 figures, the model JSON and the reference cases — **plus byte-identical copies** into `app/src/main/assets/` and `app/src/test/resources/`
Android (on-device)	`FutureRiskFeatures` (22 features), `FutureRiskModel` (loader + Platt calibration + factor ranking), `FutureRiskAnalyzer` (bands, staleness, insufficient data), `FutureRiskReporter` (a throttled summary), `FutureRiskCardModel` / `FutureRiskCard`, wired through `MovementController` into `TrackingScreen`
Backend	a `RISK` payload kind with **12 validation rejections**, a new `risk_assessments` table, `/me.latestRisk`, `GET /authority/risk`, per-tourist history, and overview counts
Dashboard	a new **"Future-risk predictions"** panel, a `Risk (model)` column and a `RISK` feed filter — shown as **predictions**, never folded into the SOS queue or the incident feed

Two deliberate engineering choices worth recording:

- **Reuse, no new dependency.** Inference uses the app's **existing** `DecisionForest` JSON walker (the only change is an optional `expectedFeatures` parameter so a 22-feature model loads without weakening the existing 24-feature check). **No ML framework, runtime or serialization format was added.**
- **One sensor subscription.** The risk model is fed by the movement watch's existing accelerometer stream — no second listener, no extra battery cost — and the tourist-facing prediction therefore works **fully offline**.

14.5 Model selection under a real deployment constraint

`tune.py` searched **65 configurations** on the **validation split only**. The quality winner (300 trees, unbounded depth, validation PR-AUC **0.875909**) exports to **20,059.7 KB** and **cannot ship to a phone**. Only **1 of 65** configurations fits the 2,000,000-byte asset budget; the selected one (120 trees, depth 11, leaf 4, validation PR-AUC **0.845112**) exports to **1,968,314 bytes** and ships at **1,974,297** with its metadata. The **−0.031 PR-AUC cost is stated in the model card**, and the selection rule is recorded in `tuning.json` and **re-derived by a test** rather than living in a document.

14.6 Verification for this round (all measured)

Check	Command	Result (measured)
Android JVM suite	`:app:testDebugUnitTest --rerun`	**484 tests, 0 failures, 0 errors, 5 skipped** across **45 classes** (94 new: 93 across 8 new classes + 1 live E2E)
Backend rules (in-process)	`python -m pytest backend\tests -q`	**220 passed** (was 192; +28 for `test_risk.py`)
Backend acceptance (real subprocess)	`python backend\verify_integration.py`	**33/33 PASS** (was 26/26; the batch now also carries a `RISK` summary, so the acceptance run proves a prediction reaches `/authority/risk` and raises no SOS)
ML suite	`python -m pytest tests future_risk/tests -q` (from `ml/`)	**179 passed** (was 86; +93 for the new pipeline)
Live client -> real server	`AuthorityLiveBackendTest` with `-Dtsm.e2e.baseUrl=...`	**5/5 passed, 0 skipped** (the new one pushes an on-device prediction and proves `sosTotal` does not move)
APK build	`:app:assembleDebug`	BUILD SUCCESSFUL

Held-out test numbers (10,415 windows, 6 unseen subjects, 1,339 positive): majority baseline PR-AUC 0.128565 · logistic regression 0.659633 · **hist gradient boosting 0.815000** (better, **not shippable on-device**) · **shipped forest 0.764569 PR-AUC / 0.936848 ROC-AUC**, precision 0.6707, recall 0.7498, F1 0.7080 · **trial sensitivity 0.929577** (264 of 284 real falls flagged) · Brier 0.062600 -> **0.056675** after Platt calibration fitted on validation only · **shuffled-label control 0.477**.

APK **19.48 MB** (20,422,301 bytes), SHA-256 `AF690EB209C408F202CD8EC9CAAA9FD1350C060AFC21AF7CDEF8706940CABDBA`, copied to the project root, `Desktop` and `Downloads` as `app-debug.apk`.

14.7 Still not done here (stated, not hidden)

- **No emulator or device.** The `FutureRiskCard` on a real screen and the whole live chain (sensor -> window -> 22 features -> shipped forest -> calibrated band) remain **NOT EXECUTED** (INST-22, INST-23). The deployment *is* offline-safe by construction, and `FutureRiskReferenceTest` proves the shipped asset agrees with scikit-learn, but that is not the same as having watched it run on a phone.
- **The dashboard panel was never opened in a browser.** `app.js` is syntax-checked with `node -e`; the rows it renders come from `backend/tests/test_risk.py`, which asserts the API responses they are built from.
- **The model is not a deployment-ready alarm.** 5.09 false alarms per minute of ordinary activity (16.30/min at the raw stride), over-confident mid-range probabilities, lab data from 30 healthy adults, a one-second horizon only, and

an untested "dropped phone vs falling person" failure mode.
- **The four named datasets still cannot support a supervised incident model.** Nothing here changes that; the future-risk model uses a fifth dataset with real outcome labels, and says so everywhere.

---

15. Self-service tourist registration: the tourist creates their own account (2026-10-18)

Until this round every tourist account had to be created **by an officer** at the dashboard, and the tourist then signed in with an id and a one-time password. That is the wrong shape for the thing being built: a tourist who installs the app, with nobody at a desk, could not use the service at all. This round adds the one journey that was missing — **the tourist registers themselves** — and makes the result show up on the authority's screen with no officer action.

15.1 What "register" had to mean

Three constraints shaped the design:

- the Tourist ID is the account's permanent name and has to look like one (`TR-2026-00124`), and the tourist has to be able to **write it down** — so it must be shown, and it must be copyable;
- it must be **unique by construction**, not by a client-side counter that two phones can race;
- the PIN is a credential: hashed like a password, never stored in the clear, never returned, never logged.

15.2 The ID is minted by the database, not the client

A client-side counter is the classic bug here: two phones registering in the same second both compute "00124". The allocation instead happens in SQLite:

- a new table, `tourist_id_sequence (id INTEGER PRIMARY KEY AUTOINCREMENT, issued_at INTEGER NOT NULL)`, where each registration inserts one row and reads back the number SQLite issued;
- the text id is `TR-<current UTC year>-<5-digit zero-padded number>`, and `tourists.id` is the **primary key**, so even a text collision cannot create a duplicate — the code steps over to the next number;
- the counter is **never reset**, across years or restarts. The year is for a human to read, not a reset boundary.

`backend/tests/test_registration.py` registers twelve tourists in a row and asserts twelve distinct ids; it also forces a collision and asserts the code moves to the next number rather than failing.

15.3 The PIN is the credential; the response carries no secret

The `tourists` table has `NOT NULL` `password_salt` / `password_hash` columns from the officer-created flow. Registration keeps that schema (no migration risk to existing rows) by writing a **randomly generated secret that is immediately discarded** into those columns; the account's real credential is the PIN, run through the project's existing scrypt `hash_password`. Neither the PIN, nor its salt, nor its digest is ever written to a response or a log — `test_registration.py` asserts the response contains the id and the account's public fields and nothing that looks like a secret, and `test_security.py`'s rule (no hash or salt leaves the server) is unchanged.

The response to a successful registration is deliberately **only**:

```
{"tourist": {"touristId": "TR-2026-00124", "displayName": "…", "phone": "…",
             "registeredAtMillis": 1792305616556, "registrationSource": "SELF",
             "emergencyContactCount": 1},
```

```
  "note": "Registration complete. Sign in with this Tourist ID and the PIN you chose. …",
  "serverTimeMillis": 1792305616556}
```

No token: **registering does not sign you in.** The tourist signs in as a separate step, exactly so the id on the success screen is something they must keep.

15.4 One public write, and the privilege escalation it refuses

`POST /api/v1/auth/register` is the only new route and the only public **write**. It is fenced by reading as little as possible:

- it creates only `tourists` rows;
- it reads **no privilege field**. A body carrying `"role": "authority"` is refused with `400`, and there is no field that could grant one — the escalation path does not exist rather than being blocked;
- it issues no session, so it cannot be used to obtain a token;
- it is idempotent *per person*, not per call: a phone number that is already registered is `409`.

The validation is pure and lives in one place, `backend/tsm/registration.py`, reusing the existing `hash_password` (scrypt) and the same contacts validator the outbox uses (`payloads.normalise("CONTACTS", …)`), so a contact accepted here is a contact the sync path would accept.

15.5 The app: register -> get the id -> sign in

The Authority screen's sign-in card now has two modes, deliberately separate — *Sign in* and *Register as tourist* — because one creates the account and the other uses it. Registering collects a name, a phone number, a chosen PIN and its confirmation, and any number of emergency contacts (added one at a time, each with a name, a number and an optional relationship; removable). The form is validated **on the phone first** so a mistyped confirmation is answered instantly instead of costing a round-trip; the server validates everything again and its `409` is shown as "*already registered*" rather than as a generic failure.

On success the screen becomes a **registration-success card** showing the Tourist ID in a monospace readout with a **Copy Tourist ID** button, the account's name and phone, and a plain-language instruction on how to sign in next — including that the PIN is not shown again and that a forgotten PIN means an officer must reset it. *Continue to sign in* carries the id into the sign-in field and clears the PIN fields.

New code: `authority/RegistrationForm.kt` (pure validation), `AuthorityContract.kt` / `AuthorityApi.kt` / `AuthorityClient.kt` (request and response codec), `AuthorityViewModel.kt` (the state machine and the "a later failure must not erase an id still being copied" rule), and `ui/screens/AuthorityScreen.kt` (the mode toggle, the form and the success card).

15.6 The dashboard: Registered Tourists, with registration separate from tracking

`GET /api/v1/authority/tourists` was enriched (additively — nothing existing changed shape) and the dashboard's "Tourists" panel became **"Registered Tourists"**, driven from it. Each row now carries:

`touristId`, `displayName`, `phone`, `registeredAtMillis`, **`registrationStatus`**, `registrationSource` (`SELF` for an app registration, the officer's id for a desk account, `null` for older rows), **`trackingStatus`**, `lastSeenMillis`, `hasPosition`, `latestPosition`, `emergencyContactCount`, `openSosCount`, `latestRiskBand`, `latestRiskAtMillis`, `state`.

The point of the split is that three different facts stay three different facts:

- **registration** — *is this a real account?* (`REGISTERED`);
- **tracking** — *have we heard from the phone?* (`AWAITING_FIRST_REPORT`, later `OK` / `STALE`);

- **safety** — the open SOS count and the latest risk band, which were already there.

A tourist who has registered but never reported reads **"Location not yet available"**; the panel never shows a position that does not exist. New registrations appear at the top of the list on the next refresh (the list is newest-first) with no officer action.

15.7 A latent bug found on the way

While adding the prediction's contract to the register, the `RISK` ingest was found to insert its `risk_assessments` row **outside** the `with db().write()` block that inserted its `events` row — so the prediction and its audit entry could commit separately. That is invisible in the happy path and wrong exactly when it matters (a crash between the two writes would leave a prediction with no audit trail). It now runs inside the same transaction. The prediction itself was unchanged.

15.8 Verification for this round (all measured)

Check	Command	Result (measured)
Android JVM suite	`:app:testDebugUnitTest --rerun`	**515 tests, 0 failures, 0 errors, 6 skipped** across **47 classes** (31 new/expanded: 13 `RegistrationFormTest` + 11 `AuthorityViewModelTest` + 4 `AuthorityApiTest` + 2 `AuthorityHttpServerTest` + 1 live E2E)
Backend rules (in-process)	`python -m pytest backend\tests -q`	**265 passed** (was 220; +45 for `test_registration.py`)
Backend acceptance (real subprocess)	`python backend\verify_integration.py`	**41/41 PASS** (was 33/33; +8: self-register, the id is minted, no token/secret, sign in with id + PIN, duplicate phone 409, role escalation 400, appears on the officer's register with no position, survives a restart)
ML suite	`python -m pytest tests future_risk/tests -q` (from `ml/`)	**179 passed** (unchanged)
Live client -> real server	`AuthorityLiveBackendTest` with `-Dtsm.e2e.baseUrl=...`	**6/6 passed, 0 skipped** — the new case registered `TR-2026-00001`, signed in with that id and the chosen PIN, read the row back from the officer's authenticated API, then delivered a real-GPS position and watched the row become `trackingStatus=OK`, `hasPosition=true`
APK build	`:app:assembleDebug`	BUILD SUCCESSFUL

The live run logged the dashboard row before and after the report, which is the clearest evidence of the split:

```
[e2e] registered TR-2026-00001 for +91 81792305616395
[e2e] signed in as the freshly-registered TR-2026-00001
[e2e] dashboard row after registration:  {… "registrationStatus":"REGISTERED",
     "registrationSource":"SELF", "trackingStatus":"AWAITING_FIRST_REPORT",
     "hasPosition":false, "latestPosition":null, "state":"STALE" …}
[e2e] dashboard row after a real report: {… "registrationStatus":"REGISTERED",
     "trackingStatus":"OK", "hasPosition":true,
     "latestPosition":{"latitude":18.5204,"longitude":73.8567,"simulated":false}, "state":"OK" …}
```

APK **19.74 MB** (20,701,949 bytes), SHA-256 `564B1C726F8A5D6ECB16225949EAAA1D0F71C0EA1DA6D164B523F2A118DD470C`, copied to the project root, `Desktop` and `Downloads` as `TouristSafetyMonitoring-2026-10-18-self-registration-debug.apk`.

15.9 Still not done here (stated, not hidden)

- **The registration UI was never rendered on a screen** (INST-24). The form logic is JVM-tested (`RegistrationFormTest`, `AuthorityViewModelTest`), the wire format is real-socket-tested (`AuthorityHttpServerTest`), and the whole journey runs through the **production** client against a real server on the JVM (§7.1 of the test report) — but no Compose screen was drawn, because there is still no emulator or device in this environment.
- **The dashboard panel was not opened in a browser.** `app.js` is checked with `node --check` and its inputs are asserted in `test_registration.py`, but `renderTourists` was not watched rendering real rows.
- **Nothing was committed.** This project has no version control in this environment; every change lives in the working tree.
- **The PIN is the only credential for a self-registered account.** There is no self-service PIN reset (by design — the PIN is irrecoverable, and the supported reset is an officer action), and no rate limiting on the public registration route beyond the phone-number uniqueness check. Both are documented scope lines rather than oversights.

---

16. Round: two-role sign-in, and the submit button that was dead on a fresh install

16.1 The report

A phone with a fresh install of the previous APK opened on the **Authority** screen showing **NOT SIGNED IN**, with **Sign in with PIN** and **Create my account** both greyed out. Nothing on the screen said why, and no Tourist ID was ever minted — so registration was, from the user's side, impossible.

16.2 The root cause, exactly

`AuthorityScreen.kt` enabled all three of *Save address*, *Sign in* and *Register* with

```
enabled = !busy && address.isNotBlank()
```

`address` was seeded from the stored session's `baseUrl`, which is **empty on a fresh install**. So the first thing a new user met was three dead buttons whose only requirement was a backend address that the app never asked for and never explained. It was not a registration bug, not a credential bug and not a network bug: the **submission controls were gated on a field the tourist was never meant to fill in**.

Two structural problems fell out of the same design:

- the tourist's sign-in/registration and the *operator's* backend configuration lived on one screen; and
- the operator's screen was reachable from the tourist's path at all.

16.3 What changed

1. **The gate moved to form validity.** `RegistrationForm.fieldErrors(...)` reports a problem **per field**; `RegistrationForm.outstanding(...)` lists only what is genuinely missing; the button is live whenever a submission

is not in flight. Pressing it with a bad form shows the named problems. **No submit control anywhere depends on a backend address.**

2. **The address left the tourist's path.** It now lives in *Settings → Backend connection (advanced)*, where **Test connection** performs a real `GET /api/v1/health` — *Connected* is claimed only when something actually answered, and an address with nothing behind it reports *Offline*.
3. **A build-time default, seeded exactly once.** `-PAUTHORITY_URL` (or `TSM_AUTHORITY_URL`, or `AUTHORITY_URL=` in `local.properties`) becomes `BuildConfig.DEFAULT_AUTHORITY_URL`; `AppContainer.start()` writes it through `seedBaseUrlIfUnset`, which is a no-op once an address exists. This pass's APK carries `http://192.168.1.102:8080`, so a first launch on this machine's Wi-Fi is already pointed at the server.
4. **Two roles, two sessions.** The app opens on a **Welcome** screen offering *I'm a tourist* and *I'm an authority officer*; a returning tourist with a valid session is offered *Continue* above the choices. Officer and tourist sessions are stored under **separate keys** in the same DataStore, so an officer sign-in cannot make the phone behave like a tracked tourist and either sign-out leaves the other alone. Both are restored on restart.
5. **The boundary is the server's.** The officer console reads `GET /api/v1/authority/tourists`, which already required `require_auth("authority")`. Hiding a screen proves nothing, so the live test presents a **tourist** token to that route and asserts **403**.
6. **The backend configuration screen** survived as the advanced/technical screen (address, connection state, queue + *upload now*, tourist profile, PIN change, the authority's view of this phone), with sign-in and registration removed from it.

16.4 A refactor with a reason

`DataStoreAuthoritySessionStore` took a `Context` only to derive a default `DataStore`, which made it untestable off-device. It now takes the `DataStore` directly, with `createAuthoritySessionStore(context)` as the production factory. That is what turns "the session survives a restart" and "the officer session is separate" from asserted-by-inspection into **asserted**: `AuthoritySessionStoreTest` drives a real `PreferenceDataStore` over a throw-away file on the plain JVM, with no Robolectric. The file's handle is released by joining the cancelled scope before a second store opens the same file — without that the test would be racy.

16.5 Verification for this round (all measured)

Check	Command	Result (measured)
Android JVM suite	`:app:testDebugUnitTest --rerun`	**545 tests, 0 failures, 0 errors, 7 skipped** across **49 classes** (+30: 10 `OfficerViewModelTest`, 8 `AuthoritySessionStoreTest`, 5 `RegistrationFormTest`, 4 `AuthorityApiTest`, 2 `AuthorityHttpServerTest`, 1 live E2E)
Backend rules (in-process)	`python -m pytest backend\tests -q`	**265 passed** (unchanged — no backend file changed)
Backend acceptance (real subprocess)	`python backend\verify_integration.py`	**41/41 PASS** (unchanged)
ML suite	`python -m pytest ml\tests ml\future_risk\tests -q`	**179 passed** (unchanged)

Check	Command	Result (measured)
Live client -> real server	`AuthorityLiveBackendTest` with `-Dtsm.e2e.baseUrl=http://127.0.0.1:8080`	**7/7 passed, 0 skipped** — the new case signed in as an officer, read the register through the app's own console method (every row asserted to carry an id and both statuses, and to have a position only when it claims one), then presented a **tourist** token to the same route and received **HTTP 403**
APK build	`:app:assembleDebug -PAUTHORITY_URL=http://192.168.1.102:8080`	BUILD SUCCESSFUL — 17 dex entries; `192.168.1.102:8080` and `DEFAULT_AUTHORITY_URL` found together in `classes8.dex`. **19.80 MB** (20,759,379 bytes), SHA-256 `CBA40235472992A30AA51F56ED8839F24DBF71C3094D7D06B8A316C34FC2CD79`

16.6 Still not done here (stated, not hidden)

- **The new screens were never rendered on a device** (INST-24, INST-25): the Welcome role chooser, the tourist sign-in / create-account screen with its success card and copy button, and the officer console. Their logic is JVM-tested (49 classes), the register route is real-socket-tested in both directions, and both client methods were driven against a real server on the JVM — but no Compose screen was drawn, because there is still no emulator or device in this environment (`adb devices` is empty).
- **The 403 is proven at the client and server level, not from a device.** The device's own `HttpURLConnection` and its loopback/Wi-Fi routing remain unexercised (INST-21).
- **No rate limiting was added** to the officer sign-in route. Officer ids are generated (not chosen) and the modules are unique, so enumeration is not a practical attack, but a lockout after repeated failures would still be an improvement.
- **Nothing was committed.** There is no version control in this environment; every change lives in the working tree.

---

17. Round: the phone could never reach the authority — Android's cleartext block (2026-10-18)

17.1 The report

Two screenshots from a real phone, both taken on the phone itself, both showing a red banner:

- **Tourist sign-in:** "*The authority at http://192.168.1.102:8080 is not answering. Check that the server is running and that this phone is on the same Wi-Fi network.*" — with the connection card reading **Status: Offline**, "*No answer from http://192.168.1.102:8080.*"
- **Authority officer:** "*Cannot reach http://192.168.1.102:8080: **Cleartext HTTP traffic to 192.168.1.102 not permitted***".

The second banner is the tell: it is not a message this app writes. It is the wording Android itself uses when the **platform refuses to send a plain-HTTP request**.

17.2 The root cause, exactly

- The authority is the project's own server: plain HTTP on port 8080 over the LAN (`http://192.168.1.102:8080`). There is no TLS.
- The app sends it with `HttpURLConnection` (`HttpUrlConnectionAuthorityHttp`), and the transport wraps any `IOException` as `AuthorityCall.Unreachable`, whose message is shown verbatim as "*Cannot reach* : ".
- **Android 9 (API 28) and later refuse cleartext HTTP by default** for any app whose `targetSdk` is 28 or higher. This app's `targetSdk` is **35**, and the manifest declared **neither** `android:usesCleartextTraffic` **nor** `android:networkSecurityConfig`.

So the request was killed by the network-security policy *before it reached the network*. The server was never asked, never answered, and never logged anything — which is exactly what the authority's own access log shows: across the whole session there is not one request from the phone's address, only the host's own browser on the dashboard. The tourist screen's "is not answering" is the same failure with the platform's message summarised away; the officer screen happened to print the raw sentence.

This had been true since the first build. It is not a regression from the two-role round, and no amount of restarting `run.py` would ever have changed it.

17.3 The fix

1. **`app/src/main/res/xml/network_security_config.xml`** (new) — a `<base-config cleartextTrafficPermitted="true">` with the reasoning written down in the file, so the next reader does not "harden" it back into a broken state. The file also says what to do if the authority ever gains TLS.
2. **`AndroidManifest.xml`** — the `<application>` now declares `android:networkSecurityConfig="@xml/network_security_config"`.
3. **`NetworkSecurityConfigTest`** (new, 3 tests) — reads the shipped manifest and the profile and fails if the manifest stops pointing at a profile, if the profile stops permitting cleartext, or if the manifest re-blocks it with `usesCleartextTraffic="false"`. A build setting that silently disables the app's only way to reach its backend deserves a test, not a comment.

Nothing else changed: the address resolution, the tourist/officer sessions, the officer endpoint and the `require_auth("authority")` boundary from round 16 are untouched.

17.4 Verification for this round (all measured)

Check	Command	Result (measured)
Android JVM suite	`:app:testDebugUnitTest --rerun`	**548 tests, 0 failures, 0 errors, 7 skipped** across **50 classes** (+3 `NetworkSecurityConfigTest`; the 7 skips are the opt-in live-backend tests)
The shipped policy itself	`aapt2 dump xmltree` on the built APK	`<application>` carries `android:networkSecurityConfig(0x01010527)=@0x7f0c0002`, and `aapt2 dump resources` resolves `0x7f0c0002` to `res/xml/network_security_config.xml`, which contains `cleartextTrafficPermitted` — i.e. the fix is **inside the artefact**, not just the source

Check	Command	Result (measured)
Backend rules (in-process)	`python -m pytest backend\tests -q`	**265 passed** (unchanged — no backend file changed)
Backend acceptance	`python backend\verify_integration.py`	**41/41 PASS** (unchanged)
ML suite	`python -m pytest ml\tests ml\future_risk\tests -q`	**179 passed** (unchanged)
APK build	`:app:assembleDebug -PAUTHORITY_URL=http://192.168.1.102:8080`	BUILD SUCCESSFUL — 17 dex entries; `192.168.1.102:8080` still in `classes8.dex`. **20.33 MB** (21,317,914 bytes), SHA-256 `F0786379EC1917F9004E03F72E9317FB0C0F97B4B7D4BF9F312A911D3BE5D239` (copies re-published to the project root, Desktop and Downloads)
Live client -> real server	`AuthorityLiveBackendTest`	**NOT re-run this round** — the backend was stopped at the user's request, so the seven opt-in cases skip (that is what `skipped="7"` above records). Their last measured result is round 16's **7/7**.

17.5 Still not done here (stated, not hidden)

- **The fix has not been seen working on the phone.** The two screenshots are the first real-device evidence this project has produced, and they show the *old* APK failing. Installing the new APK and watching the banner clear is a manual, on-device step (INST-24/INST-25 remain NOT EXECUTED). What *is* proven is that the artefact now contains the policy Android requires.
- **The old APK must be replaced.** The phone still has the build without this change; the address it was seeded with is retained by an in-place `adb install -r` because the debug signing key and package are unchanged.
- **The server must be running for the phone to connect**, and the phone must be on the same Wi-Fi. The fix removes the *platform* block; it does not make the backend start itself. A Windows Firewall inbound allow rule for `python.exe` already exists on this machine, so no firewall change was needed.
- **Nothing was committed.** There is no version control in this environment; every change lives in the working tree.